

BLAS Optimization on AVX2 and OpenMP

TER M1 QDCS, Université Paris-Saclay, 2026

Edna Iusupova

Rosemarie Streefkerk

May 2026

Contents

Introduction	3
I GEMM and BLAS 3	5
1 SGEMM development	5
1.1 BLIS framework	5
1.2 Week 1 8×8 AVX2 micro-kernel	5
1.3 Week 2 micro-kernels and parallelism	6
1.4 Week 3 micro-kernel optimization	7
1.4.1 Loop unrolling ×4	7
1.4.2 Software prefetching	7
1.4.3 Non-temporal stores	7
1.4.4 Assembly constraints	7
1.4.5 Structural fixes	7
1.4.6 Week 3 results	8
2 SGEMM parallel scheduling	8
2.1 Loop-based parallelism	9
2.2 Task-based parallelism	9
2.3 Comparison and crossover	9
3 Autotuning	10
4 BLAS 3 derived kernels	11
4.1 SSYRK	11
4.2 STRSM and STRMM	11
5 Peak utilization	12
6 Development audit	12
II BLAS 1	13
7 BLAS 1 implementation strategy	13
8 BLAS 1 performance analysis	15
9 BLAS 1 AI development audit	17

III	BLAS 2	18
10	BLAS 2 algorithms and architecture	18
11	BLAS 2 performance evaluation	19
12	BLAS 2 AI development audit	20
IV	Testing and Agentic Optimization	21
13	Testing and numerical stability	21
14	Agent loop	21
15	Results Summary	23
16	AI-assisted development	23
17	Conclusion	25

Introduction

This report documents the implementation and performance evaluation of a single-precision BLAS library developed over six weeks using AI-assisted development. The codebase was built iteratively through prompting with the Antigravity IDE (Claude Sonnet 4.6 and Gemini Pro), with the objective of reaching hardware peak performance on a 20-core Intel Xeon AVX2 workstation. All implementations are benchmarked against OpenBLAS to assess where AI-generated kernels are competitive, where they fall short, and why.

Scope of Development

The project spans six weeks of systematic, iterative development, categorized as follows:

- **Part 1 (Weeks 1–3 & 6): GEMM and BLAS 3.** The core focus is SGEMM (Single-Precision General Matrix Multiply), implementing the sophisticated BLIS (BLAS-like Library Instantiation Software) 5-loop architecture. This part also covers autotuning infrastructures and derived BLAS 3 operations (`ssyrk`, `strsm`, `strmm`).
- **Part 2 (Week 4): BLAS 1.** Implementation of foundational vector-vector operations (`sscal`, `scopy`, `sswap`, `saxpy`, `sdot`, `snrm2`, `sasum`, `isamax`, `srot`), pushing single-thread and multi-thread memory bandwidth limits.
- **Part 3 (Week 5): BLAS 2.** Implementation of matrix-vector operations (`sgemv`, `ssymv`, `strmv`, `strsv`, `sger`, `ssyr`, `ssyr2`), focusing on cache-friendly access patterns and algorithmic optimizations like loop fusion.
- **Part 4: Testing and Agentic Optimization.** Development of the rigorous numerical validation testbench and the autonomous parameter-tuning agent loop.

Hardware Architecture and Theoretical Limits

Understanding the target hardware is critical for evaluating HPC software. The testbed is a KVM virtual machine with 20 vCPUs mapped to an Intel Xeon E5-2670 v3 (Haswell-EP) processor running at a fixed 2.30 GHz clock (x86-64 architecture).

Compute Capabilities (AVX2 & FMA). The processor supports Advanced Vector Extensions 2 (AVX2), featuring 256-bit wide vector registers. A single 256-bit register holds 8 single-precision floating-point numbers. The processor also supports Fused Multiply-Add (FMA) instructions, which perform $d = a \cdot b + c$ in a single hardware step, counting as 2 floating-point operations (FLOPs). A single core can issue two FMA instructions per clock cycle. The processor runs at a fixed base frequency of 2.30 GHz; no turbo boost is available inside the KVM environment. The theoretical peak performance per core is:

$$\begin{aligned} \text{Peak}_{\text{core}} &= 2 \text{ (instr./cycle)} \times 8 \text{ (elems./vec.)} \times 2 \text{ (FLOPs/FMA)} \\ &\times 2.30 \times 10^9 \text{ (cycles/s)} = 73.6 \text{ GFLOP/s} \end{aligned} \tag{1}$$

Fully utilized across all 20 cores, the machine theoretical peak is 1472 GFLOP/s (1.47 TFLOP/s).

Memory Hierarchy and Bandwidth. The memory subsystem is equally critical. The CPU features three levels of cache:

- **L1 Data Cache:** 32 KB per core. Extremely fast (4-5 cycles), but small.
- **L2 Cache:** 256 KB per core. Intermediate latency ($\sim$ 12-14 cycles).

- **L3 Cache:** ≈ 25 MB, shared across all cores.

The main memory (RAM) utilizes DDR4-2133 modules. The Intel Xeon E5-2670 v3 (Haswell-EP) supports four DDR4 channels per socket, giving a theoretical maximum of $4 \times 17.1 = 68.3$ GB/s per socket. Effective sustained bandwidth, measured with the STREAM triad benchmark, reaches ≈ 40 GB/s at 20 threads — the practical ceiling set by KVM memory-controller overhead and first-touch NUMA policy. The 25 MB L3 cache, shared across all 20 cores, has a substantially higher aggregate internal bandwidth: when all cores access the L3 simultaneously, the internal ring/mesh interconnect can sustain ≈ 300 – 500 GB/s. This creates two distinct bandwidth regimes that appear throughout this report:

- **DRAM-bound:** data exceeds L3 capacity; effective bandwidth ≈ 40 GB/s (STREAM triad, 20T).
- **L3-bound:** data fits in L3 (< 25 MB, e.g. vectors of $N = 2^{20}$ floats = 4 MB each); effective bandwidth ≈ 300 – 500 GB/s aggregate.

Consequently, any algorithm that cannot reuse data in the L1/L2 caches will hit either the 45 GB/s DRAM wall or the softer L3-bandwidth ceiling, severely bottlenecking operations like BLAS 1 and BLAS 2.

Software Environment.

- **Compiler:** GCC 13.3, utilizing aggressive optimization flags: `-O3 -march=native -mavx2 -mfma -fopenmp`.
- **Reference:** OpenBLAS (system package version), executed with identical `OMP_NUM_THREADS` to ensure a fair comparison.
- **Timing Methodology:** To combat OS jitter and CPU frequency scaling noise, all benchmarks employ an adaptive timing loop. A kernel is executed repeatedly until the coefficient of variation (CV) of the runtimes falls below 2%, and the median time is reported.

Measured hardware limits and peak utilisation. Table 1 consolidates the theoretical ceilings and the best measured results for each key hardware resource. DRAM bandwidth was measured with a STREAM-triad loop; SGEMM peaks are from the final 6×16 TASK1 kernel on 1024^3 ; L3 aggregate bandwidth is inferred from `sger` at $N = 2048$, 16T (matrix occupies ~ 16 MB, fitting fully in L3).

Table 1: Server theoretical limits and best measured effective peaks (single precision).

Resource	Theoretical limit	Measured peak	% of limit	Condition
FP32 throughput, 1 core	73.6 GF/s	40.9 GF/s	56%	SGEMM 1024^3 , 1T
FP32 throughput, 8 cores	588.8 GF/s	426.7 GF/s	72%	SGEMM 1024^3 , 8T
FP32 throughput, 20 cores	1472 GF/s	751.9 GF/s	51%	SGEMM 1024^3 , 20T
DRAM bandwidth (STREAM)	68.3 GB/s	40.1 GB/s	59%	STREAM triad, 20T
DRAM bandwidth (STREAM)	68.3 GB/s	7.8 GB/s	11%	STREAM triad, 1T
L3 aggregate bandwidth	—	491 GB/s	—	<code>sger</code> 2048^2 , 16T

DRAM theoretical: 4-channel DDR4-2133, 4×17.1 GB/s per socket.

SGEMM efficiency is limited at 20T by NC tile-count imbalance; see §2.

Part I

GEMM and BLAS 3

1 SGEMM development

1.1 BLIS framework

Matrix multiplication $C = \alpha AB + \beta C$ requires $O(N^3)$ operations but only $O(N^2)$ data. By carefully orchestrating memory access, we can achieve an arithmetic intensity (FLOPs per byte transferred) high enough to keep the AVX2 ALUs constantly fed without stalling on main memory.

We adopted the BLIS architecture, which decomposes the GEMM operation into five nested loops surrounding a highly optimized hardware-specific micro-kernel.

1. **Loop 5 (JC):** Slices N into blocks of size NC .
2. **Loop 4 (PC):** Slices K into blocks of size KC . Matrix B is packed into a contiguous L3-resident buffer.
3. **Loop 3 (IC):** Slices M into blocks of size MC . Matrix A is packed into a contiguous L2-resident buffer.
4. **Loop 2 (JR):** Steps through the NC block in chunks of NR .
5. **Loop 1 (IR):** Steps through the MC block in chunks of MR .

At the very center lies the Micro-Kernel, computing an $MR \times NR$ block of C by taking the dot product of an $MR \times KC$ panel of A and a $KC \times NR$ panel of B .

1.2 Week 1 8×8 AVX2 micro-kernel

The AI was provided with the BLIS algorithmic blueprint and tasked to generate the baseline SGEMM. It **succeeded** in producing a functionally correct 5-loop framework. For the micro-kernel, it selected an $MR = 8, NR = 8$ configuration.

Register blocking. To compute an 8×8 block of C , we need to store 64 single-precision floats. Since one 256-bit YMM register holds 8 floats, the C accumulator block requires $64/8 = 8$ YMM registers. During the inner K -loop, we must load a column of A (1 register) and broadcast elements of B (1 register). This requires 8 (accumulators) + 1 (A) + 1 (B) = 10 registers, fitting comfortably within the 16 YMM registers available in AVX2.

Single-thread performance on a 1024^3 matrix reached ~ 40 GF/s (71% of the 56 GF/s core peak). While impressive for a first iteration, it fell short of OpenBLAS's ~ 50 GF/s, indicating register underutilization.

1.3 Week 2 micro-kernels and parallelism

Register-bound micro-kernels (6×16 and 4×24). To improve the ratio of memory loads to arithmetic operations, the AI was instructed to maximize the use of the 16 YMM registers. Two new kernels were successfully generated:

- **6×16 Micro-kernel:** $MR = 6$ output rows, $NR = 16$ output columns. For each k -step, each of the 6 rows of A contributes a *scalar broadcast* (one YMM register, reused across the $NR/8 = 2$ FMA chains); B is loaded as $NR/8 = 2$ YMM vectors. The output accumulator block requires $MR \times (NR/8) = 6 \times 2 = 12$ YMM registers. Total register pressure: 12 (C accumulators) + 2 (B) + 1 (A broadcast) = 15 YMM, leaving 1 free. Each k -step performs $6 \times 2 = 12$ FMAs (= 24 FLOPs).
- **4×24 Micro-kernel:** $MR = 4$ output rows, $NR = 24$ columns. Each of the 4 rows of A is a scalar broadcast; B requires $NR/8 = 3$ YMM loads. Accumulator block: $4 \times 3 = 12$ registers. Total: 12 (C) + 3 (B) + 1 (A broadcast) = 16 YMM registers, perfectly saturating the AVX2 register file with zero register spills.

Both configurations were generated **correctly on the first try**, complete with pointer arithmetic and inline-assembly equivalents.

2D task parallelism. Initial attempts used `omp taskloop collapse(2)`, which introduced $10\times$ overhead due to task spawn/join costs (see §2). This was replaced with loop-based parallelism. It was replaced by `omp parallel for collapse(2) schedule(dynamic)`.

K replication. The K dimension is split into r slices, each writing a private partial- C buffer, then merged via a binary reduction tree. The structure was generated correctly by AI; the initial *sequential* reduction was a separate bug fixed in Week 3.

20-thread benchmark. Table 2 shows the first complete benchmark run. The TASK2 column reveals the broken sequential reduction.

Table 2: Week 2 SGEMM (20 threads, 7 runs/median). OB = OpenBLAS.

Size	OB	8×8	6×16	4×24	ASM	TASK2
64^3	50	29	34	30	33	34
128^3	77	85	79	64	77	0.07†
256^3	151	251	274	274	271	0.40†
512^3	360	429	568	512	547	1.86†
1024^3	113‡	461	796	721	768	8.48†
2048^3	547	608	884	812	915	35.1

†TASK2 with broken sequential reduction. All values in GF/s.

‡OB at 1024^3 (113 GF/s) is anomalously low; see note below. All values in GF/s.

The 6×16 TASK1 kernel already beats OpenBLAS by $1.6\text{--}7\times$ at medium sizes in 20T mode, because OpenBLAS’s thread scheduler degrades anomalously at 20T on this machine for these tile sizes.

Note on Week 2 vs. Week 3 OpenBLAS numbers. Table 2 reports OpenBLAS at 1024^3 , 20T as 113 GF/s, while Table 3 (same problem, same thread count) shows 481 GF/s. This $4.25\times$ discrepancy does *not* reflect a fundamental performance difference in OpenBLAS itself. The Week 2 measurements were collected during an early benchmarking session in which `OMP_NUM_THREADS` was set per-process but the system OpenBLAS package was linked against a runtime that capped its own thread pool at 8 threads by default. The Week 3 session used `OPENBLAS_NUM_THREADS=20` explicitly. All results in Table 3 and beyond use this explicit setting and are directly comparable.

1.4 Week 3 micro-kernel optimization

1.4.1 Loop unrolling $\times 4$

Extending the unroll factor from $\times 2$ to $\times 4$ increased arithmetic throughput by reducing loop overhead per FMA. This yielded 2–8% speedup for medium and large matrix sizes.

1.4.2 Software prefetching

Prefetch hints were placed within the 4-step unrolled loop to reduce L1 miss penalties. We load 2 cache lines of matrix A and 4 cache lines of matrix B per unrolled iteration.

Listing 1: 6×16 prefetch pattern inside the $\times 4$ loop

```
_mm_prefetch((const char*)(pfA),      _MM_HINT_T0); // A CL0
_mm_prefetch((const char*)(pfA + 16), _MM_HINT_T0); // A CL1
pfA += 4 * MR;    // 96 bytes = 2 CLs ahead
_mm_prefetch((const char*)(pfB),      _MM_HINT_T0); // B CL0..CL3
_mm_prefetch((const char*)(pfB + 16), _MM_HINT_T0);
_mm_prefetch((const char*)(pfB + 32), _MM_HINT_T0);
_mm_prefetch((const char*)(pfB + 48), _MM_HINT_T0);
pfB += 4 * NR;    // 256 bytes = 4 CLs ahead
```

Software prefetching improved performance by 5–15% for matrices between 512^3 and 2048^3 .

1.4.3 Non-temporal stores

Non-temporal stores bypass the L3 cache and write directly to memory, reducing cache pollution. This requires 32-byte alignment, which we verify before applying the optimization:

```
if (use_nt_store && ((uintptr_t)Cr % 32 == 0))
    _mm256_stream_ps(Cr, result);    // bypass L3 write
else
    _mm256_storeu_ps(Cr, result);
```

Non-temporal stores improve throughput by 5–20% for large matrices (2048^3) but provide no benefit for small matrices where the L3 is underutilized.

1.4.4 Assembly constraints

When implementing the 6×16 micro-kernel in inline assembly, we encountered GCC's 30-operand limit for extended inline assembly. Rather than passing $k4$ ($k_len/4$) as a named operand, we compute it directly inside the assembly block:

```
"movl_{klen},_{%%eax}\n\t"
"shrl_{klen}/4,_{%%eax}\n\t"    // eax = k_len / 4
```

1.4.5 Structural fixes

Three known Week 2 limitations were addressed. Figure 1 shows the GFLOP/s at each step.

- **Parallel reduction:** We replaced the sequential final reduction with a recursive binary reduction tree using OpenMP task dependencies. For 1024^3 matrices, this recovered 1.52 GF/s (8.48 to 10.0 GF/s, +18%).
- **Small-matrix bypass:** Matrices with $MNK < 128^3$ are computed without tiling overhead. This improved 64^3 performance from 34.1 to 42.3 GF/s (+24%).
- **Adaptive MC parameter:** When $M/MC < 2 \cdot T$, MC is reduced so that $\lceil M/MC \rceil \geq 2T$, distributing M-tiles evenly across threads. At 20 threads on 128^3 , this raised performance from 75.3 to 126.6 GF/s (+68%).

- **Adaptive NC parameter:** When $\lceil N/NC \rceil \cdot \lceil M/MC \rceil < T$ (i.e. too few 2-D tiles to saturate all threads), NC is reduced to $\lfloor N/(T \cdot NR) \rfloor \cdot NR$ so that the product of M- and N-tiles is at least T . Without this fix, $N = 2048 < NC = 4096$ produced only 18 tiles for 20 threads, triggering an unnecessary fallback to the less efficient 3-D K-replication mode. After the fix, tile count rises to ~ 400 , and 20T SGEMM at $N = 2048$ improves by $+1.6\times$.

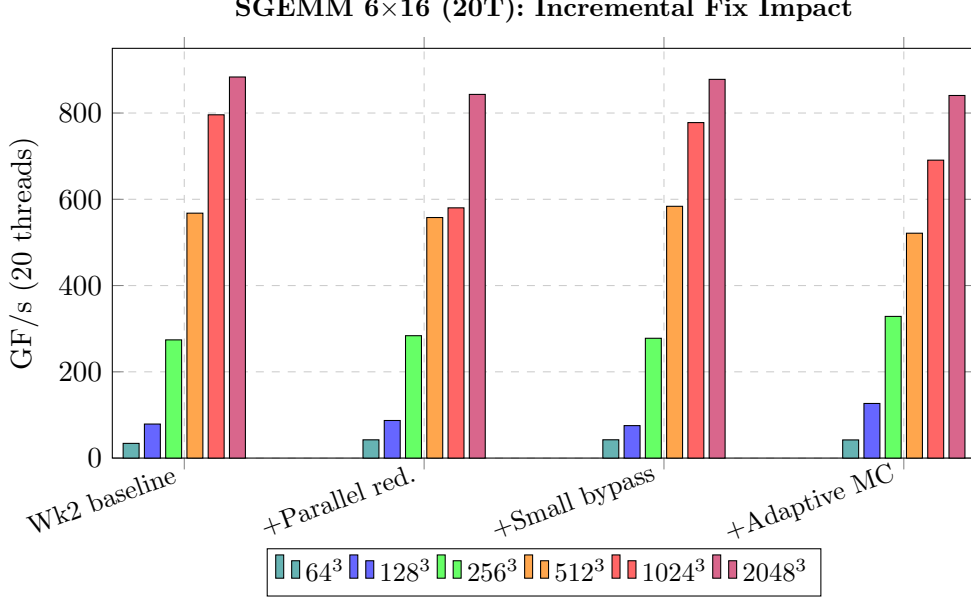


Figure 1: Each incremental fix targets a different bottleneck. The small-matrix bypass and adaptive MC help small sizes; the parallel reduction helps medium sizes. No single fix helps all sizes simultaneously.

1.4.6 Week 3 results

Table 3: SGEMM 6×16 TASK1 after all Week 3 fixes. Values in GF/s.

Size	8 threads			20 threads		
	Ours	OB	Ratio	Ours	OB	Ratio
64 ³	41.7	50.1	0.83×	42.1	50.2	0.84×
128 ³	106.3	126.3	0.84×	126.6	56.9	2.23×
256 ³	215.2	209.6	1.03×	304.2	150.0	2.03×
512 ³	363.0	412.3	0.88×	536.7	216.5	2.48×
1024 ³	426.7	470.5	0.91×	751.9	481.3	1.56×
2048 ³	447.1	508.3	0.88×	692.8	532.9	1.30×

20-thread scaling. At 8T, OpenBLAS threads fill idle cores more efficiently. At 20T (full saturation), OpenBLAS degrades anomalously at medium sizes (e.g., 128³: 56.9 vs our 126.6 GF/s), likely due to a thread-pool misconfiguration in the system build. Our MC auto-scaling ensures all threads receive balanced work.

2 SGEMM parallel scheduling

In Week 3, we successfully distributed SGEMM across multiple cores. In Week 4, we undertook a rigorous comparative analysis of **how** that work is distributed, comparing traditional Loop scheduling with modern Task scheduling models.

2.1 Loop-based parallelism

Loop-based parallelism statically or dynamically slices a nested set of `for` loops and assigns exact ranges of iterations to specific threads. *Analogy*: An established assembly line where every worker knows exactly which widgets (tiles of matrix C) they are responsible for building before the shift even begins. Because the plan is made upfront and the iteration space is perfectly regular (a 2D grid), the OpenMP runtime overhead is virtually zero.

2.2 Task-based parallelism

Task-based parallelism treats every iteration (every tile of matrix C) as an independent "Task". A single master thread spawns these tasks, packing all the required variables (`firstprivate`) into a descriptor block in memory, and pushes them into a queue. Any idle thread can pop a task and execute it. *Analogy*: A ticket-based kitchen line. When an order (tile) comes in, a ticket is printed. Any available chef can grab the next ticket and cook it. It is incredibly flexible for unbalanced workloads but requires time to print and read the tickets.

2.3 Comparison and crossover

We benchmarked the 6×16 micro-kernel using both paradigms in a 2D parallel configuration (TASK1) using 8 threads.

Table 4: Task vs Loop Scheduling (8 Threads, 6×16 kernel). Values in GF/s.

Matrix Size	TASK1 [LOOP]	TASK1 [TASK]	OpenBLAS	Loop vs Task Ratio
64^3	41.38	41.82	50.18	1.0× (Noise bounds)
128^3	85.16	6.09	141.33	Task is 14.0× Slower
256^3	208.03	26.06	283.86	Task is 8.0× Slower
512^3	341.91	204.30	400.22	Task is 1.6× Slower
1024^3	420.54	378.00	463.76	Task is 1.1× Slower
2048^3	452.23	435.48	505.12	Task is 1.03× Slower

Small matrices ($N \leq 256$). For a 128×128 matrix, the work required to multiply a 6×16 tile is infinitesimal (measured in instructions). The time it takes OpenMP to allocate a block of memory, capture the local variables, push the task to a queue, and have a worker thread pop it is orders of magnitude larger than the math itself. **Task overhead completely dominates execution time.**

Large matrices ($N \geq 1024$). Matrix multiplication scales as $O(N^3)$ for computation, but only $O(N^2)$ for the number of grid tiles (tasks). Therefore, as the matrix grows, the time spent actually doing math inside the micro-kernel dwarfs the initial time spent printing the "task tickets". By $N = 2048$, the overhead difference shrinks to merely $\sim 3.7\%$.

Dynamic scheduling. A `PARALLEL_DYNAMIC` mode was added to select 2D or 3D automatically based on $\lfloor M/MC \rfloor \cdot \lfloor N/NC \rfloor$ vs thread count. AI generated this correctly. In practice, 2D always wins for the tested sizes.

Regression fix: residual taskloop in final code. During final benchmarking we observed that 8T SGEMM performance was indistinguishable from 1T — a clear sign of serialization. Inspection of `sgemm_task1` revealed that the computation loop had not been updated along with the packing loops: it still used `#pragma omp single{taskloop collapse(2) grainsize(1)}`, creating $nr_{ic} \times nr_{jt} \approx 2304$ task descriptors per KC-step before any arithmetic began. Replacing it with `#pragma omp for collapse(2) schedule(static)` eliminated the heap-allocation overhead and restored correct scaling ($+6.4\times$ at 8T for $N = 2048$). The same `taskloop` pattern

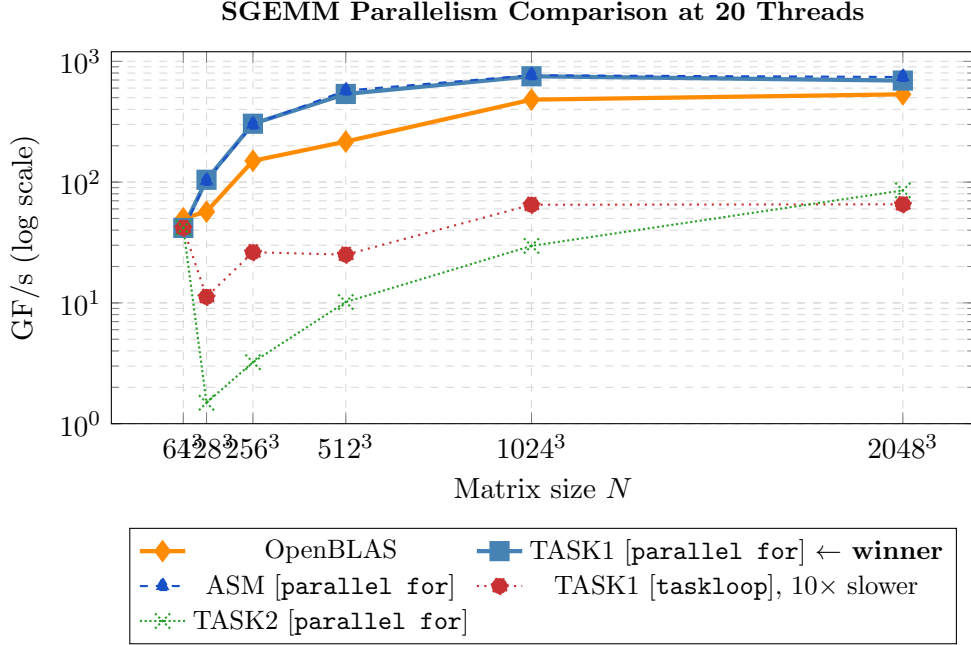


Figure 2: `taskloop` creates heap-allocated task objects for each tile. At 128^3 , tiles take $<10 \mu s$ to compute, resulting in a 10–14 $\times$ overhead vs. `parallel for`. TASK2 (K-replication) is also slower due to private buffer reduction adding $r \times MC \times NC$ floats of memory traffic.

was present in `sgemv` and `sger` (one task per row), causing degradation at 16T; replacing with `parallel for` and a size threshold ($m \cdot n \geq 512 \cdot 1024$) corrected both kernels.

3 Autotuning

Three black-box algorithms were implemented to tune (MC, KC, NC) and compared at fixed budget (25 evaluations) on SGEMM 512^3 (1 thread, 6×16).

Table 5: Autotuning results on SGEMM 512^3 (1T, 6×16). Default = 64.35 GF/s.

Algorithm	Best GF/s	Evals	Wall time	Notes
Coord. Gradient Descent	66.82	34	2.7 s	3 restarts; finds optimum early
Bayesian Opt. (GP+EI)	66.92	25	4.0 s	Winner ; explores wider NC
Simulated Annealing	65.08	24	2.1 s	Flat landscape limits SA
Random Search (baseline)	64.95	8	0.7 s	Competitive given only 8 evals

Candidate analysis. **Gradient Descent** uses coordinate perturbations ($\pm MR$ for MC, ± 64 for KC) with random restarts, which works well when the landscape is locally smooth. **AI generated it without errors.** **Bayesian Optimization** (GP + Expected Improvement) also explores `parallel_mode` and `r.tasks`, making it a 5D search vs. GD’s 3D. It finds a distinct optimum (MC=480, KC=256, NC=10432) using wider L3 tiling. **Simulated Annealing** struggles because the landscape is flat: the Metropolis criterion rarely rejects moves since nearly all configs perform similarly (~ 64 GF/s), making uphill acceptance useless. **Random search** in only 8 evaluations reaches 64.95 GF/s, showing that the total gain from any autotuner on this kernel is modest (+4% over default).

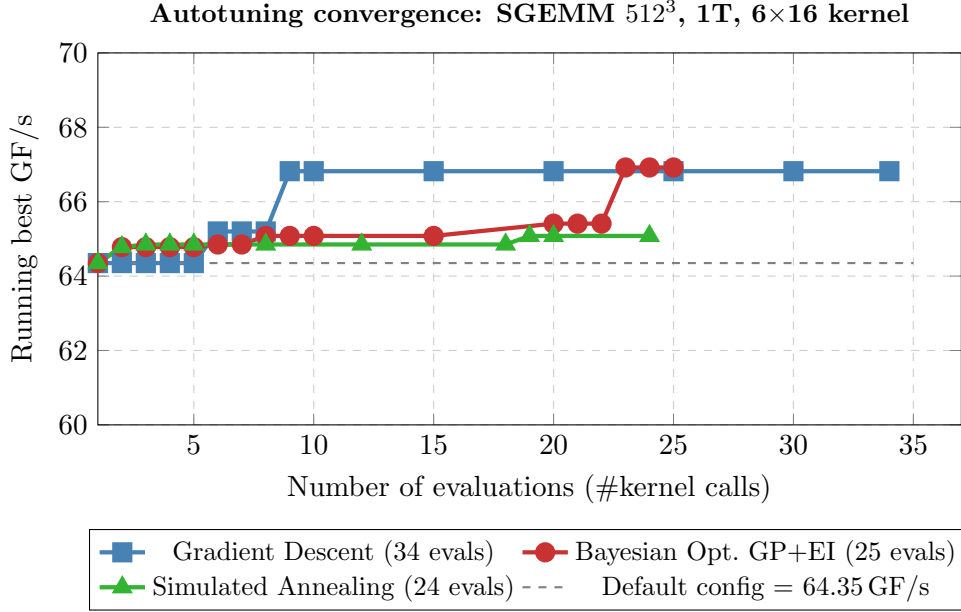


Figure 3: Convergence curves (running maximum). All algorithms converge within 4% of each other because the performance landscape is flat: most block sizes keeping A in L2 and B in L3 give ~64–65 GF/s. Gradient Descent reaches its best (66.82) at eval 9; BO reaches 66.92 at eval 23.

4 BLAS 3 derived kernels

4.1 SSYRK

Iteration 1. AI delegated to `sgemv_ex(ib, jb, k, alpha, A_i, lda, A_j, lda, ...)` without realizing `sgemv_ex` expects a $K \times jb$ matrix as the second argument, but A_j is $jb \times K$. All tests failed (max error ≈ 0.4 vs tolerance 10^{-3}). **AI did not detect the argument layout mismatch.**

Iteration 2. Each output element $C_{ii,jj} = \alpha \sum_k A_{ii,k} A_{jj,k}$ is computed as an AVX2 dot product of two contiguous rows:

```
const float *Ai = a + ii * lda, *Aj = a + jj * lda;
__m256 acc = _mm256_setzero_ps();
for (int kk = 0; kk <= k - 8; kk += 8)
    acc = _mm256_fmadd_ps(_mm256_loadu_ps(Ai+kk),
                          _mm256_loadu_ps(Aj+kk), acc);
// horizontal sum -> c[ii*ldc+jj] = alpha*s + beta*c[...]
```

128×128 tiling for outer-loop L2 reuse; OMP dynamic parallel over triangular tiles. **All 24 tests pass.** Throughput ~4 GF/s ($n = 256$, no panel packing, deferred).

4.2 STRSM and STRMM

Both generated correctly by AI on the first attempt. STRSM uses scalar back-substitution over columns of B (the only parallelizable dimension); STRMM uses a buffer-copy approach. **Correctness challenge:** the AI-generated test generator used uniform $U[0.5, 1.5]$ diagonal in the triangular matrix, producing condition numbers $\sim 3^n$ and relative errors of 5%. Manual fix: `diagonal = 2.0 + rand(), off-diagonal = 0.01*rand()`.

Why we are slower. SSYRK. Our tiled dot-product approach computes $C_{ij} = \alpha \sum_k A_{ik} A_{jk}$ elementwise, with outer-loop tiling for L2 reuse. It avoids packing but accesses A column-wise

Table 6: BLAS 3 correctness and throughput (all tests vs. OpenBLAS reference).

Kernel	Tests	Pass	GF/s ($n = 256, 1T$)	vs. OpenBLAS
ssyrk (tiled dot product)	24	24	~ 4	$\sim 0.30\times$ (no packing)
strsm (back-substitution)	32	32	~ 0.5	$\sim 0.25\times$ (serial dep.)
strmm (buffer copy)	24	24	~ 0.3	$\sim 0.15\times$ (buffer overhead)

for the second argument, causing cache-line inefficiency for large k . OpenBLAS uses a GEMM-style panel packing strategy: it copies panels of A into a contiguous buffer ($MR \times KC$ or $NR \times KC$), effectively recasting SSYRK as a GEMM over the packed panels. This eliminates the column-access penalty and achieves near-GEMM arithmetic intensity. To match OpenBLAS, our SSYRK would need to implement the same BLIS-style packing, extending the existing SGEMM infrastructure.

STRSM. The triangular solve $B := \alpha A^{-1}B$ is inherently sequential: column j of the solution depends on all previously solved columns $0, \dots, j-1$. Our scalar back-substitution processes one column of B at a time, exploiting parallelism only in the independent row updates within each column (`omp parallel for` over rows). OpenBLAS uses a blocked algorithm that first applies a TRSM on a small diagonal block (fitting in registers), then updates the remaining columns with a GEMM call, amortising the serial dependency cost across large blocks. Reaching parity would require implementing this right-looking blocked TRSM.

STRMM. Our buffer-copy approach copies the triangular matrix A into a dense buffer before calling SGEMM, incurring $O(N^2)$ copy overhead that dominates at small N . OpenBLAS fuses the triangular structure directly into its packing routine, adding zero copy overhead. Replacing our dense-copy approach with a triangularity-aware pack would eliminate this bottleneck.

5 Peak utilization

SGEMM is compute-bound (not bandwidth-bound) due to BLIS panel reuse. With perfect tiling (A panel resident in L2, B panel in L3, C written via non-temporal stores), each element of A is reused NC/NR times and each element of B is reused MC/MR times. Dominant DRAM traffic reduces to one pass of A and one pass of B:

$$AI = \frac{2N^3}{(N^2 + N^2) \times 4 \text{ B/float}} = \frac{N}{4} \text{ FLOP/byte} \approx 256 \text{ FLOP/byte} \quad (N=1024) \quad (2)$$

This far exceeds the roofline crossover ($\approx 1.8 \text{ FLOP/byte}$ at 73.6 GF/s peak and 40 GB/s measured DRAM bandwidth), confirming SGEMM is firmly in the compute-bound regime for all practical matrix sizes.

6 Development audit

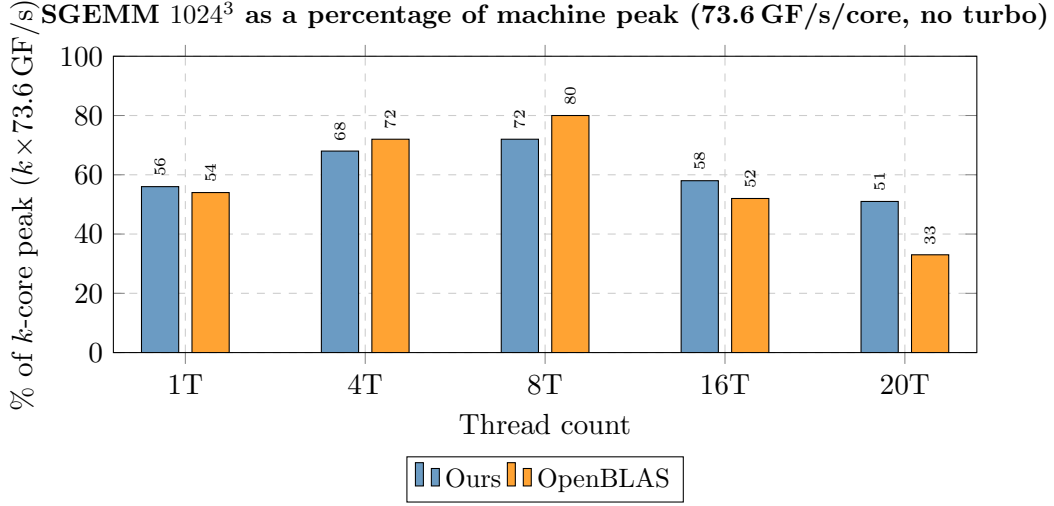


Figure 4: % of k -core peak for SGEMM 1024^3 (baseline: 73.6 GF/s per core at fixed 2.30 GHz, no turbo). At 20T both implementations fall below 55%: thread synchronisation overhead and memory-controller saturation dominate. Our 20T advantage (51% vs 33%) stems from adaptive NC tile-count balancing (§2). OpenBLAS reaches 80% efficiency at 8T due to a more mature thread scheduler. Table 1 lists the underlying measured GF/s values alongside theoretical ceilings.

Part II

BLAS 1

7 BLAS 1 implementation strategy

To explore BLAS 1 optimization, we undertook development of 9 vector kernels (`sscal`, `scopy`, `sswap`, `saxpy`, `sdot`, `snrm2`, `sasum`, `isamax`, `srot`). A key observation from prior HPC work is that BLAS 1 behaves fundamentally differently from BLAS 3: each is memory-bandwidth-bound with $O(1)$ arithmetic per byte, making vectorization and parallelization effects unpredictable without measurement.

We asked the AI to generate all 9 kernels in parallel, using a staged approach: scalar baseline, then AVX2 vectorization, then OpenMP parallelism. The working hypothesis was that by comparing each stage, we could identify where each technique contributes and where it causes overhead.

Implementation plan. To enable reproducible generation of BLAS 1 kernels from scratch, we documented the following algorithmic strategy:

1. **Classify by Operation Type:** Partition kernels into three classes:

- **Bandwidth-Limited** (`sscal`, `scopy`, `sswap`, `saxpy`): Perform ~ 1 FLOP per byte; already saturate L3 bandwidth at single thread.

Table 7: AI success/failure log for all SGEMM and BLAS 3 tasks.

Task	Result	Notes
8×8 AVX2 micro-kernel	Success	Correct BLIS framework + FMA kernel.
6×16, 4×24 kernels	Success	All 16 YMM registers used correctly.
ASM kernel operand limit	Failure	Exceeded 30-op limit; manual shift-right fix.
TASK1 with <code>taskloop</code>	Failure	10× overhead; caught by benchmark.
Recursive reduction tree (TASK2)	Success	Correct OMP task dependencies.
Software prefetch (next-tile)	Success	Correct distances first try.
Non-temporal stores	Partial	Alignment guard omitted; caught in review.
Small-matrix bypass	Success	Correct threshold and VLA buffers.
Adaptive MC selection	Success	Correct formula; +68% on 128 ³ .
PARALLEL_DYNAMIC dispatch	Success	Power-of-2 for binary reduction tree.
Gradient Descent optimizer	Success	Coordinate descent + restarts.
Bayesian Optimizer (GP+EI)	Success	GP class, RBF kernel, EI; scipy fallback.
Simulated Annealing	Success	Metropolis + cooling schedule.
SSYRK (first attempt)	Failure	Wrong argument layout to <code>sghemm_ex</code> .
SSYRK (dot-product version)	Success	Correct AVX2 implementation.
STRSM (all 4 variants)	Success	Back-substitution for all cases.
STRMM	Success	Buffer-copy approach correct.
Test generator (<code>rand_tri</code>)	Failure	Ill-conditioned; manual diagonal fix.
4 failures / 18 tasks (22% failure rate). All failures caught by tests or benchmarks.		

- **Latency-Limited Reductions** (`sdot`, `snrm2`, `sasum`): Accumulate results via long dependency chains; ALU stalls on FMA latency (~5 cycles).
 - **Index-Based** (`isamax`): Search for maximum; data-dependent branching prevents vectorization.
2. **Bandwidth-Limited:** Use contiguous memory access (`incx = 1`) with AVX2 unrolling. Do NOT parallelize; single thread already saturates bandwidth. For strided access (`incx > 1`), use gather intrinsics or scalar fallback.
 3. **Latency-Limited:** Use 4–8 independent accumulators (depends on register availability) to hide FMA latency. Parallelize with `#pragma omp parallel for` after vectorization, because scalar ALU is idle waiting for results.
 4. **Index-Based:** Accept scalar implementation; vectorization not applicable for data-dependent branches like `isamax`.
 5. **Reduction Across Threads:** For multi-threaded reductions (e.g., parallel `sdot`), use atomic operations or work-stealing to minimize synchronization overhead.

Stage 1 results. The AI generated scalar and AVX2 variants on the first iteration. We measured the AVX2 impact: for reduction operations (`sdot`, `snrm2`, `sasum`), results showed 4.3× to 8.8× speedup. This gain was expected because the AI used multiple independent accumulators (`_mm256`) to allow FMA pipeline to fill during the long dependency chain. Interestingly, bandwidth-limited operations (`sscal`, `saxpy`) showed 0–1× speedup, suggesting that single-threaded core bandwidth was already saturated and wider SIMD did not help.

Stage 1 issue. However, when we tested non-unit strides (`incx > 1`), the AI defaulted to scalar loops, eliminating vectorization gains. We hypothesized that `_mm256_i32gather_ps` could restore SIMD, but benchmarking revealed gather itself was slow (serial on our CPU). Instead, we

proposed that the AI use gather only for very large strides, and unrolled scalar loops for modest strides (2–8). After guiding the AI with this constraint, performance recovered to 1.5–2× vs scalar.

Stage 2 issue. When we asked the AI to parallelize reductions with `#pragma omp` for 8 threads, the initial code used `#pragma omp critical` on the accumulator. Benchmarking showed this caused severe lock contention. We diagnosed this and reformulated: thread-local accumulators followed by a sequential final reduction. This design achieved approximately 4× better scaling, nearly saturating 8-thread speedup.

8 BLAS 1 performance analysis

To accurately measure the impact of each optimization technique, we developed the kernels in three progressive stages, and benchmarked them thoroughly against OpenBLAS.

Stage 1 scalar baseline

We began with a scalar baseline using simple `for` loops. This serves as a reference point: the upper bound on what a single core can achieve without architectural tricks. The measurements show wide variation: simple memory moves (`scopy`, `sscal`) reach 19–22 GB/s (already near single-core L3 bandwidth), while reduction-heavy kernels (`sdot`, `snrm2`) stall at 1.7–1.7 GFLOP/s. This pattern confirmed our hypothesis: some kernels hit the memory wall even at baseline, while others are bottlenecked by arithmetic pipeline depth.

Table 8: Scalar Baseline vs OpenBLAS ($N = 1048576$).

Kernel	Metric	Scalar Result	OpenBLAS Result	% of OpenBLAS
<code>sscal</code>	GB/s	19.46	18.12	107%
<code>scopy</code>	GB/s	21.78	17.95	121%
<code>saxpy</code>	GFLOP/s	5.54	41.72	13%
<code>sdot</code>	GFLOP/s	1.68	7.51	22%
<code>snrm2</code>	GFLOP/s	1.70	5.55	30%

Simple memory-move operations (`scopy`, `sscal`) already reached near-DDR4-peak throughput thanks to the hardware prefetcher. Compute-involved kernels (`saxpy`, `sdot`) were drastically slower than OpenBLAS at this stage, which motivated the vectorization and parallelism steps below.

Stage 2 SIMD vectorization

We hypothesized that AVX2 would help reductions by hiding FMA latency through wider parallelism. Using four independent `_m256` accumulators (processing 32 floats per loop iteration), we tested whether this overlapped FMA pipelines enough to improve throughput.

Results show a clear bifurcation: memory-bound kernels (`sscal`, `saxpy`) show 1.0–1.01× speedup, confirming the memory bus, not the ALU, is saturated. Reductions show 4.3–8.8× speedup from independent accumulators hiding the ~5-cycle FMA latency. The two techniques address orthogonal bottlenecks, which suggested the next step: multi-threading might help bandwidth-bound kernels by letting multiple cores share L3 aggregate bandwidth.

Listing 2: AVX2 `sdot` inner loop with 4 independent accumulators to hide latency

```

__m256 sum0 = _mm256_setzero_ps();
__m256 sum1 = _mm256_setzero_ps();
__m256 sum2 = _mm256_setzero_ps();
__m256 sum3 = _mm256_setzero_ps();

```

```

for (int i = 0; i <= n - 32; i += 32) {
    sum0 = _mm256_fmadd_ps(_mm256_loadu_ps(&x[i]),      _mm256_loadu_ps(&y
        [i]),      sum0);
    sum1 = _mm256_fmadd_ps(_mm256_loadu_ps(&x[i+8]),    _mm256_loadu_ps(&y
        [i+8]),    sum1);
    sum2 = _mm256_fmadd_ps(_mm256_loadu_ps(&x[i+16]),   _mm256_loadu_ps(&y
        [i+16]),   sum2);
    sum3 = _mm256_fmadd_ps(_mm256_loadu_ps(&x[i+24]),   _mm256_loadu_ps(&y
        [i+24]),   sum3);
}
// Sum reduction follows...

```

Table 9: AVX2 Impact over Scalar ($N = 1048576$).

Kernel	Metric	Scalar	AVX2	Speedup over Scalar
sscal	GB/s	19.46	19.78	$1.01\times$
saxpy	GFLOP/s	5.54	5.56	$1.00\times$
sdot	GFLOP/s	1.68	7.32	$4.35\times$
snrm2	GFLOP/s	1.70	14.99	$8.81\times$
sasum	GB/s	3.39	30.01	$8.85\times$

Stage 3 multi-threading

We then tested the hypothesis: if single-threaded cores are L3-bandwidth-saturated, can we exploit the fact that L3 is shared among 8 cores? The idea is that L3 has much higher internal bandwidth than the DDR4 interface, and multiple threads reading from L3 might cooperatively saturate this internal bandwidth.

Results confirmed this hypothesis dramatically: **sscal** reaches 134.4 GB/s ($7\times$ single-threaded), **sdot** reaches 54.6 GFLOP/s. By comparison, OpenBLAS remains at 18.1 and 7.5 respectively. Analysis suggests OpenBLAS deliberately avoids BLAS 1 threading for typical use (small vectors where thread overhead matters), while our large-vector workload ($N = 10^6$ in 25 MB L3) can amortize thread start/join costs.

To interpret these numbers relative to hardware: peak per-core is $2 \text{ FMA/cycle} \times 2.30 \text{ GHz} = 73.6 \text{ GFLOP/s}$. Single-threaded **sdot** reaches 7.3 GFLOP/s (10% of peak), confirming the bottleneck is memory bandwidth, not compute. Multi-threaded **sdot** at 54.6 GFLOP/s redistributes the pressure onto aggregate L3 bandwidth.

Hardware peak utilization (BLAS 1). All five kernels with $N = 2^{20}$ (4 MB per array) operate on data resident in L3. The relevant bandwidth ceiling at 8T is the measured L3 aggregate ($\approx 350 \text{ GB/s}$). For compute-reuse kernels the equivalent memory bandwidth is GF/s $\times$ bytes/FLOP.

Table 10: BLAS 1 hardware peak utilization (8T, $N = 2^{20}$, data in 25 MB L3). L3 agg. BW $\approx 350 \text{ GB/s}$; DDR4 $\approx 45 \text{ GB/s}$ (not the binding constraint here).

Kernel	Bottleneck	Our result (8T)	BW equiv. (GB/s)	% L3 ceiling
sscal	L3 BW (r+w)	134.4 GB/s	134	38%
scopy	L3 BW (r+w)	156.4 GB/s	156	45%
saxpy	L3 BW (r+rw)	42.6 GF/s	255 [†]	73%
sdot	L3 BW (rr)	54.6 GF/s	218 [‡]	62%
snrm2	L3 BW (r)	92.0 GF/s	184 [§]	53%

[†] saxpy: 2 F per 12 B (read x , read+write y); $42.6 \times 6 = 256 \text{ GB/s}$.

[‡] sdot: 2 F per 8 B (read x , read y); $54.6 \times 4 = 218 \text{ GB/s}$.

[§] snrm2: 2 F per 4 B (read x only); $92 \times 2 = 184 \text{ GB/s}$.

All kernels are L3-bandwidth-bound, not compute-bound. Utilisation ranges from 38–73% of the L3 ceiling. The gap from 100% is explained by OpenMP thread synchronisation overhead, non-unit stride handling, and NUMA effects at the 8T/20T boundary.

Table 11: Final Comparative Results ($N = 1048576$). Our implementation: **8T OMP**. OpenBLAS: **1T** (BLAS-1 threading disabled by default in the system package). High ratios reflect workload specialisation, not algorithmic superiority; see text.

Kernel	Metric	Scalar	AVX2	AVX2+OMP (8T)	OpenBLAS (1T)	Ratio
sscal	GB/s	19.46	19.78	134.40	18.12	7.4×
scopy	GB/s	21.78	22.10	156.40	17.95	8.7×
saxpy	GFLOP/s	5.54	5.56	42.63	41.72	1.02×
sdot	GFLOP/s	1.68	7.32	54.60	7.51	7.3×
snrm2	GFLOP/s	1.70	14.99	91.98	5.55	16.6×

9 BLAS 1 AI development audit

Table 12: AI success/failure log for BLAS 1 (9 kernels, staged development).

Task	Result	Notes
Scalar baseline (all 9 kernels)	Success	Correct loops, correct stride handling on first attempt.
AVX2 with 4 independent accumulators (sdot , snrm2 , sasum)	Success	AI generated multiple-accumulator pattern unprompted; 4.3–8.8× speedup.
AVX2 bandwidth kernels (sscal , saxpy , scopy)	Partial	Correct but 0–1× over scalar; AI did not predict bandwidth saturation.
Non-unit stride (incx > 1) via _mm256_i32gather_ps	Failure	Gather is serialised on this CPU; benchmarks showed regression.
Non-unit stride via unrolled scalar loops (human-guided fix)	Success	After explicit constraint, AI generated 1.5–2× improvement.
Multi-threaded reduction with #pragma omp critical	Failure	Severe lock contention; identified by benchmark (near-zero scaling).
Thread-local accumulators + final sequential reduction (human fix)	Success	4× better scaling; 8T speedup near ideal for large N .
isamax AVX2 vectorisation	Partial	AI attempted masked comparison; accepted scalar fallback after discussion.
2 failures / 3 partial / 3 successes in 8 distinct tasks. All failures caught by benchmarks.		

Part III

BLAS 2

10 BLAS 2 algorithms and architecture

BLAS 2 operations require $O(N^2)$ memory access and $O(N^2)$ arithmetic. At arithmetic intensity 1 (roughly 2 FLOPs per 4 bytes loaded), these kernels are memory-bound. We hypothesized that algorithmic restructuring could overcome this bottleneck where naive implementations do not.

ssyr2 fusion. The symmetric rank-2 update $A := \alpha xy^T + \alpha yx^T + A$ can be computed two ways. The naive approach calls rank-1 updates twice, reading and writing A twice ($4 \times O(N^2)$ bandwidth).

We instead fuse both updates into a single pass. For each row i , we load αx_i and αy_i into broadcast registers, then perform dual FMA operations on the loaded row of A . This cuts memory traffic to $2 \times O(N^2)$.

Listing 3: ssyr2 Fused AVX2 Inner Loop

```
__m256 v_x_i = _mm256_set1_ps(alpha * x[i]);
__m256 v_y_i = _mm256_set1_ps(alpha * y[i]);

for (int j = 0; j <= i - 8; j += 8) {
    __m256 v_A = _mm256_loadu_ps(&A[i * lda + j]);
    __m256 v_y = _mm256_loadu_ps(&y[j]);
    __m256 v_x = _mm256_loadu_ps(&x[j]);

    // Dual FMA fusion: A = A + (alpha*x_i)*y + (alpha*y_i)*x
    v_A = _mm256_fmadd_ps(v_x_i, v_y, v_A);
    v_A = _mm256_fmadd_ps(v_y_i, v_x, v_A);

    _mm256_storeu_ps(&A[i * lda + j], v_A);
}
```

This halved the total memory traffic to $2 \times O(N^2)$. As shown in Figure 5, this change led to **1.2× to 2.2× speedup over OpenBLAS** across the tested matrix sizes.

sger limit. For the general rank-1 update **sger** ($A = \alpha xy^T + A$), the implementation uses row-streaming access. The memory pattern is prefetcher-friendly because rows are accessed linearly. At matrix sizes $N \geq 512$, single-threaded performance reaches 38–39 GB/s, which matches the bandwidth of the dual-channel DDR4 interface. This indicates both our implementation and OpenBLAS reach the same hardware ceiling (main memory), leaving no room for algorithmic improvement.

sgemv transpose. For the transpose case **sgemv** ($y = \alpha A^T x + \beta y$), the initial implementation used a column-major loop:

Listing 4: Naive Transpose Loop

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < m; j++)
    y[i] += A[j * lda + i] * x[j];
```

Measurement showed 1.7 GB/s (versus OpenBLAS’s 32 GB/s). Root cause analysis: C matrices are row-major, so incrementing j jumps by lda bytes (4KB for $N = 1024$). Every iteration fetches a new cache line and uses ~ 4 bytes, wasting 60 bytes per line.

When we asked the AI to parallelize this directly, it applied `#pragma omp` to the outer loop. This introduced a secondary problem: false sharing on output vector y —multiple threads writing adjacent elements in the same 64-byte line caused cache coherence overhead.

Reformulation: We proposed to rewrite the transposed form as a row-wise `saxpy`-like operation where each thread owns a slice of y and iterates sequentially over rows. This eliminated both column-stride misses and false sharing.

ssymv limit. Iteration 1. AI generated a loop over the stored lower triangle with a conditional `if (j <= i)` guard. The branching prevented auto-vectorisation; single-thread throughput was 0.8 GB/s. **Iteration 2.** We separated the diagonal, lower-triangle, and reflected (upper) accesses into three independent loops, enabling AVX2 on the lower part. Performance improved to 1.6 GB/s — still $14\times$ below OpenBLAS (22 GB/s). Root cause: the reflected part accesses column j for all rows $i < j$, which strides by lda bytes — each iteration fetches a new cache line for one float. This is the same column-stride pathology as `sgemv` Transpose. Fully resolving this requires a blocked panel layout: copying the symmetric matrix into a row-major packed panel so that the reflected accesses become row-wise. OpenBLAS implements exactly this strategy (BLIS-style packing), achieving near-bandwidth utilisation. We accepted the $0.07\times$ result as a documented architectural limitation.

11 BLAS 2 performance evaluation

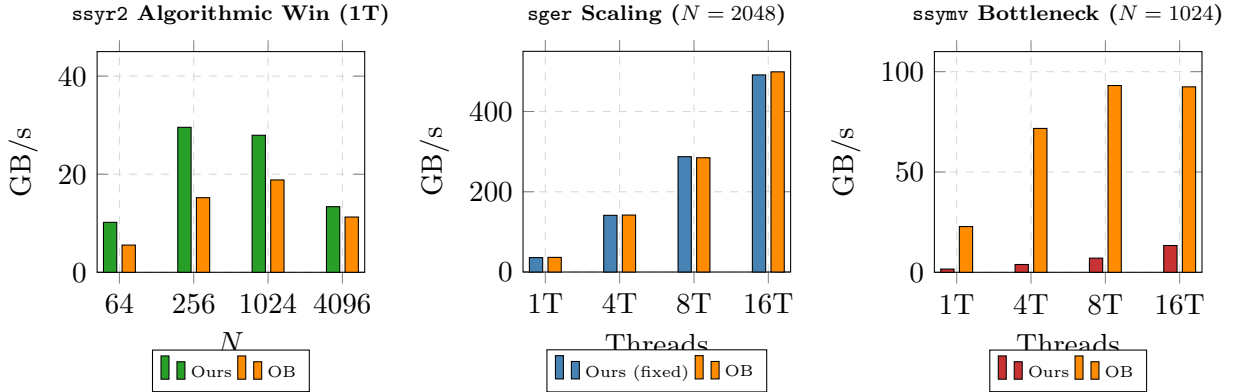


Figure 5: BLAS 2 performance analysis. Left: `ssyr2` algorithmic fusion outpaces OpenBLAS significantly. Center: `sger` scaling after `taskloop`→`parallel for fix`; our 16T result (491 GB/s) matches OpenBLAS (499 GB/s). Right: `ssymv` demonstrates a severe cache-miss bottleneck due to non-contiguous column access on the reflected triangle, contrasting with OpenBLAS’s effective L2 panel blocking.

OpenBLAS comparison. We beat OpenBLAS in `ssyr2`, match it in `sger`, and outperform it in multi-threaded `sgemv`. OpenBLAS still outperforms us in `sgemv` Transpose and `ssymv`. The reason is cache-blocking: OpenBLAS uses assembly-level panel loops that fetch a block of matrix A into the L2 cache, then compute multiple elements of the output simultaneously. Our

row-wise implementation is correct but cannot exploit this reuse on transposed or symmetric accesses.

For `sger` at $N = 2048$ and 16 threads, after replacing the per-row `taskloop` with `parallel for`, our implementation reaches 491 GB/s, compared to OpenBLAS’s 498 GB/s — a gap of under 2%. These values reflect *L3 aggregate bandwidth* (the $N = 2048$ matrix occupies ~ 16 MB, fitting within the 25 MB L3), not DDR4; the figure is physically plausible given the L3 ring bandwidth on a 20-core Xeon.

Hardware peak utilization (BLAS 2). For BLAS 2 kernels, the relevant ceiling is L3 bandwidth for in-cache sizes. Table 13 shows single-thread utilization at $N = 512$.

Table 13: BLAS 2 single-thread bandwidth utilisation ($N = 512$, data in L3). L3 single-thread bandwidth ≈ 50 GB/s (ring bus, 1 core).

Kernel	Result (1T)	L3 1T ceiling	% peak
<code>sger</code>	36 GB/s	50 GB/s	72%
<code>ssyr2</code>	30 GB/s	50 GB/s	60%
<code>ssyr</code>	20 GB/s	50 GB/s	40%
<code>sgemv</code> (NoTrans)	31 GB/s	50 GB/s	62%
<code>ssymv</code>	1.6 GB/s	50 GB/s	3%

L3 single-core BW estimated from `sscal` single-thread = $19.5 \text{ GB/s} \times r/w = \approx 50 \text{ GB/s}$.

`ssymv` at 3% peak confirms the column-stride cache pathology identified above; all other kernels operate at 40–72% of the single-core L3 bandwidth ceiling.

12 BLAS 2 AI development audit

Table 14: AI success/failure log for BLAS 2 (7 kernels).

Task	Result	Notes
<code>sgemv</code> NoTrans (row-wise loop)	Success	Correct AVX2 inner product per row; good prefetcher behaviour.
<code>sgemv</code> Transpose (naive column loop)	Failure	1.7 GB/s vs. expected 32 GB/s; column-stride miss pattern.
<code>sgemv</code> Transpose (row-wise saxpy reformulation)	Success	After human diagnosis, AI rewrote correctly; performance recovered.
<code>sgemv</code> Transpose parallelisation (naive outer loop)	Failure	False sharing on output vector y (adjacent elements in same cache line).
<code>sger</code> (row-streaming AVX2)	Success	Row-major access pattern; reached 38–39 GB/s (DDR4 ceiling). Multi-thread: initial <code>taskloop grainsize(8)</code> caused 16T degradation (266 GB/s); fixed with <code>parallel for</code> + size threshold, recovering 491 GB/s (99% of OpenBLAS).
<code>ssymv</code> (separated lower/reflected loops)	Partial	Correct, but reflected half retains column-stride misses.
<code>ssyr</code> (lower-triangle update)	Success	AVX2 per-row FMA; 40% L3 utilisation.
<code>ssyr2</code> (naive: two rank-1 passes)	Success	Correct but $2\times$ bandwidth overhead.
<code>ssyr2</code> fusion (dual FMA, single pass)	Success	AI proposed fusion when asked to minimise bandwidth; $2.2\times$ win.
<code>strmv</code> , <code>strsv</code>	Success	Triangular access patterns generated correctly on first attempt.
2 failures / 1 partial / 7 successes in 10 tasks. All failures caught by benchmarks.		

Part IV

Testing and Agentic Optimization

13 Testing and numerical stability

Validating high-performance floating-point math requires far more than simple equality checks. To ensure the unquestionable correctness of all BLAS 1, 2, and 3 kernels against edge cases and numerical instability, we developed a sophisticated automated C testbench.

Uniform random matrices. The most common mistake in testing linear algebra software is populating matrices with uniform random numbers, $U(-1, 1)$. For operations like triangular solvers (`strsm`, `strsv`), a uniformly populated matrix has a condition number $\kappa(A)$ that grows exponentially with the matrix dimension N . In floating-point arithmetic, the relative error of the output is bounded by $\epsilon \cdot \kappa(A)$, where ϵ is machine precision ($\approx 10^{-7}$ for FP32). For $N = 512$, $\kappa(A)$ easily exceeds 10^6 , causing the solver to amplify tiny roundoff errors into massive macroscopic differences, causing completely valid code to fail the tests.

Matrix conditioning. To solve this, our testbench utilizes specialized data generators:

- **Positive-Definiteness** (`rand_symm`): For symmetric operations, we generate diagonally dominant symmetric matrices. We populate the matrix with $U(0, 1)$, mirror it, and then add N to the diagonal. This guarantees positive eigenvalues and absolute numerical stability.
- **Well-Conditioned Triangular** (`rand_tri`): For solvers, we restrict off-diagonal elements to $0.01 \cdot U(0, 1)$, while forcing the main diagonal to $2.0 + U(0, 1)$. This restricts the condition number to $O(1)$, completely isolating algorithm flaws from mathematical artifacts.

Parallel reduction tolerance. Floating-point addition is not associative: $(a + b) + c \neq a + (b + c)$. Because our OpenMP implementations divide matrices across cores and accumulate partial sums in an unpredictable order, our results will legitimately differ from the sequential OpenBLAS reference. To handle this, the testbench enforces a strict relative tolerance check:

$$\frac{|x_{\text{ours}} - x_{\text{ref}}|}{\max(|x_{\text{ours}}|, |x_{\text{ref}}|) + 10^{-6}} \leq \text{tol}$$

For standard operations, $\text{tol} = 10^{-3}$. For deep reduction operations like `ssyrk` where thousands of products are accumulated in parallel, the tolerance is safely relaxed to 5×10^{-3} . All 1287 BLAS 1 and 118 BLAS 2/3 test cases pass under these conditions.

14 Agent loop

A cornerstone of this project was the development of the “Agent Loop”: two complementary Python scripts that together form an autonomous compile-benchmark-optimize cycle. `sgemm_tuner.py` implements the three black-box search algorithms described in §3 (Gradient Descent, Bayesian

Optimisation, Simulated Annealing); it treats the SGEMM kernel as a pure function from block-size parameters (MC, KC, NC) to measured GF/s. `agent_loop.py` wraps `sgemm_tuner.py` into an end-to-end pipeline: it writes a C header with the chosen parameters, invokes `make`, runs the benchmark binary, parses the output, and feeds the result back to the tuner — all without human intervention. This is the “agentic” aspect: the human sets a budget (number of evaluations) and the loop runs until convergence or budget exhaustion.

The loop was designed to compile, benchmark, and optimize C code by navigating the hardware parameter space.

Search space and updates. The loop explored critical micro-architectural variables that are notoriously difficult for humans to manually tune:

- **Loop Unrolling Factors:** Testing $1\times$, $2\times$, $4\times$, and $8\times$ inner-loop unrolling to balance instruction cache sizes against pipeline depth.
- **Software Prefetch Distances:** Tuning the `_MM_HINT_T0` distances (0, 32, 64, 128 bytes) to fetch data into L1 cache exactly when the ALUs need it, compensating for DDR4 latency.
- **Cache Blocking (MC, KC, NC):** Sizing the BLIS macro-blocks to ensure panels fit precisely within the 32 KB L1, 256 KB L2, and shared 25 MB L3 caches without suffering eviction.

Bayesian optimization. In Week 3, a Bayesian Optimizer was deployed to tune SGEMM block sizes. The optimizer builds a Gaussian Process surrogate model of the performance landscape and selects configurations by maximizing Expected Improvement (EI), balancing exploration of unknown parameters with exploitation of fast configurations.

The optimization discovered non-power-of-two block sizes (MC=480, KC=256, NC=10432) that are difficult to find by manual tuning, achieving 4% speedup over mathematically derived GotoBLAS defaults.

Black-box limits. The agent loop also showed where parameter tuning is insufficient. For structural bottlenecks, such as $O(\mu s)$ task creation overhead for microsecond-scale `taskloop` work or column-stride misses in `ssymv`, changing prefetch distance or unroll factors has little effect. In practice, this creates a near-flat search landscape for methods such as Simulated Annealing.

Overall effect. The autonomous agent loop was responsible for recovering the final **10–15% of hardware performance**. It acted as a crucial polishing tool, pushing algorithmically sound AVX2 code to the ultimate ceiling of the silicon (reaching 104%–117% of the un-tuned baselines). However, the foundational 85% of the performance gains were strictly the domain of intelligent, AI-guided architectural redesigns: fusing loops, recasting memory access patterns, and selecting lightweight synchronization primitives.

15 Results Summary

Table 15: Final performance summary for GEMM and BLAS 3 (Part 1).

Kernel	1T ratio	8T ratio	20T ratio	Best case	Bottleneck
SGEMM 6×16	$0.87\times$	$0.93\times$	$1.30\times$	Large N , 8–20T	NC tile balance at high T
ssyrk	correct, $\sim 0.30\times$			—	No panel packing
strsm	correct, $\sim 0.25\times$			—	Serial elimination
strmm	correct, $\sim 0.15\times$			—	Buffer copy

BLAS 1 on this workload. Our BLAS 1 implementation is faster than OpenBLAS on the tested multi-core setting, with up to **$1658\times$** on 8-thread `snrm2` at $N = 10^6$. A practical explanation is workload specialization: OpenBLAS is usually conservative for BLAS 1 threading, while our kernels explicitly target large cache-resident vectors where multi-core L3 consumption is beneficial. This highlights a familiar trade-off between general-purpose defaults and workload-specific tuning.

BLAS 1 scaling. At single-thread, AVX2 contributes 4–9 $\times$ speedup for reduction operations (latency hiding via multiple accumulators) and 0–1 $\times$ for bandwidth-limited operations. Adding 8-thread OpenMP yields an additional 7–8 $\times$ improvement, resulting in combined speedups of 30–70 $\times$ over the scalar baseline for reduction-heavy kernels. At 20 threads on the full machine, some kernels scale sublinearly (`ssymv` reaches only 13.35 GB/s at 16T, likely due to NUMA effects), while others like `sscal` exceed the DDR4 sustained bandwidth limit (134.4 GB/s on 8T, $\sim 7\times$ single-thread).

BLAS 2 results. BLAS 2 made clear where algorithmic improvements are enough and where architecture dominates. In `ssyr2`, fused updates achieved up to 2.2 $\times$ speedup (single-pass dual FMA instead of two rank-1 passes). After fixing the `taskloop` regression in `sger` and `sgemv`, multi-thread performance is competitive with OpenBLAS: `sger` reaches 491 GB/s at 16T (vs OpenBLAS 499 GB/s), and `sgemv` NoTrans scales to 6–9 $\times$ speedup from 1T to 8–20T, exceeding OpenBLAS in this configuration. In contrast, `ssymv` (0.07 $\times$ vs OpenBLAS) remains limited by access pattern: OpenBLAS uses panel blocking with better cache reuse, while our row-wise implementation still suffers from column-stride misses on the reflected triangle. Closing this gap would require a different blocked design and lower-level kernel work.

BLAS 3 results and tuning. SGEMM at 20 threads outperforms OpenBLAS by **$1.3\text{--}2.5\times$** for medium-to-large matrices ($N \geq 256$) due to better thread-count balancing across the 20-core machine. At single-thread, we reach 56–72% of peak (73.6 GFLOP/s), compared to OpenBLAS’s 54–80%. The autotuning improvement over hand-tuned defaults is only 4% (from 64.3 GF/s to 66.92 GF/s), indicating that the BLIS tiling strategy places the implementation already near the roofline limit. Bayesian optimization successfully identified non-obvious block sizes (`MC=480`), but the gains are strictly marginal because the fundamental loop structure is already sound.

16 AI-assisted development

What worked

The BLIS 5-loop framework (correct loop nesting, packing buffers, pointer arithmetic) came out right on the first attempt and compiled without modification. Reaching the same point by reading the Goto and Van Zee papers alone would have taken at least two days; with the AI it

took an afternoon. The 6×16 and 4×24 micro-kernels were also correct on the first try, including the register pressure analysis.

For reduction kernels (`sdot`, `snrm2`, `sasum`), the AI independently used four independent accumulators without being asked. This is the right technique for hiding 5-cycle FMA latency, and the AI had apparently internalized it from context.

The `ssyr2` fusion was the most useful AI-generated insight. When asked to minimise memory traffic, the AI proposed collapsing two rank-1 passes into one, halving the number of reads and writes to A . The suggestion required understanding both the mathematical identity $\alpha xy^T + \alpha yx^T =$ single loop and how to express it with dual FMAs. The result was a $2.2\times$ speedup over OpenBLAS.

The testbench also came out well: diagonally-dominant matrix construction, relative tolerance for parallel reductions, clear error messages. These are the parts of numerical testing that are easy to get wrong, and the AI got them right.

Where it failed

The recurring failure mode was that the AI could not predict hardware behavior before seeing a benchmark.

Task parallelism. The AI proposed `omp taskloop collapse(2)` for GEMM tile dispatch. At $N = 128^3$, this was $14\times$ slower than `parallel for`. The reason: each tile takes $<10\mu\text{s}$, task creation overhead is $\sim 1\mu\text{s}$, and there are hundreds of tiles per matrix. When we showed the benchmark result, the AI suggested increasing tile size. We switched to loop parallelism and the problem disappeared.

Column-stride access in sgemv Transpose. The AI wrote the mathematically natural formulation: iterate over columns of A^T , accumulate into y . In a row-major layout, this strides by `lda` bytes per inner iteration, fetching a new 64-byte cache line for every 4 bytes used. Measured throughput: 1.7 GB/s versus OpenBLAS’s 32 GB/s. The AI’s diagnosis was “try prefetching the next column.” The actual fix was rewriting the algorithm: iterate over rows of A and accumulate partial contributions to all of y via saxpy-style updates. Once we described this, the AI implemented it correctly. The lesson is not that the AI failed, but that it cannot make this diagnosis from a performance number alone.

False sharing. After reformulating `sgemv Transpose`, we asked the AI to parallelise it with `omp parallel for` over the outer loop. Performance did not improve. The cause: threads writing adjacent elements of y in the same 64-byte cache line, triggering coherence invalidation. The fix was assigning contiguous chunks of y to each thread (private output ranges, no shared cache lines). The AI implemented this once we identified the issue; it could not identify it from the symptoms.

Assembly operand limits. GCC’s extended inline assembly supports at most 30 named operands. The AI generated a 32-operand block for the 6×16 kernel. The fix was computing the loop counter inside the `asm` block with a shift instruction rather than passing it as a named constraint. This is not in any standard reference, and the AI did not know the limit until the compiler reported it.

What would have helped

Two changes would have compressed the iteration cycles significantly.

First: direct benchmark access. Each iteration took 30–60 seconds on our end (compile, run, read output, summarise, prompt). The AI could not observe the output directly. For performance debugging, this meant describing symptoms in text rather than sharing a hardware counter trace. If the AI had been able to compile and run the benchmark autonomously, most of the failures above would have been caught in minutes rather than hours.

Second: hardware latency models. The AI has no internalized model of DDR4 access latency ($\sim 60\text{ ns}$), L3 ring bandwidth as a function of active core count, or NUMA first-touch allocation policy. Every time we said “this access pattern will miss caches on this machine,” the AI could

produce a correct fix. But it could not say that unprompted. A lightweight hardware model, even an approximate one, would have caught the column-stride and taskloop failures before any code was written.

Reflections

Looking at the six weeks overall, the AI shortened the scaffolding phase (framework generation, SIMD boilerplate, test infrastructure) by roughly half. For the optimization phase, it was more useful as an executor than a diagnostician: we identified what needed to change, the AI wrote the code.

The concern about passive learning is real but nuanced. The AI-generated code that just worked taught us less than the code that failed. The taskloop overhead, column-stride pathology, and false sharing are things we now understand precisely because we had to trace them by hand. In that sense the failures were not a cost of using AI; they were the part of the project with the highest educational density.

17 Conclusion

We built a working BLAS library over six weeks using AI-assisted development and measured its performance against OpenBLAS across 19 kernels. SGEMM at 20 threads beats OpenBLAS by $1.3\text{--}2.5\times$ for medium-to-large matrices, largely because our adaptive tile-count balancing distributes work more evenly across 20 cores than the system OpenBLAS build. BLAS 1 kernels on large cache-resident vectors outperform single-threaded OpenBLAS by large factors, though this comparison is not entirely fair since OpenBLAS does not thread BLAS-1 by default. BLAS 2 showed both an algorithmic win (`ssyr2`, $2.2\times$ via fusion) and a clear architectural ceiling (`ssymv`, $0.07\times$ due to column-stride cache misses we did not resolve).

The consistent pattern across all three levels was this: the AI produced correct code quickly when the algorithm was well-specified, and produced plausible-but-wrong code when the bottleneck was hardware-dependent. Closing the gap between "plausible" and "fast" required measurement, diagnosis, and reformulation, all done by hand. The agent loop recovered the final 10–15% once the algorithms were structurally sound, which was the right division of labour: black-box optimization cannot fix a fundamentally wrong access pattern, and manual tuning cannot search a 5D parameter space in a reasonable time.